

# Update Summary

Date: 2026-05-11 Time: 14:26

## New Gated OUT Beliefs

- **ask-mcp-achieves-accurate-bounded-tool-use:** MCP tool integration in ask() achieves both bounded safety (iteration caps, error tolerance, transport timeouts at two layers) and accurate tool discovery (catalog reflects current server capabilities rather than a stale snapshot).
  - Blocked by: `mcp-bridge-tools-snapshot-at-connect` — The tool catalog and server instructions are populated once during `connect()` and never refreshed — there is no mechanism to pick up tools added after the initial handshake.
- **ask-mcp-tool-use-has-current-catalog:** Ask's MCP tool integration achieves full reliability — errors caught, iterations bounded — with the tool catalog always reflecting the MCP server's current capabilities rather than a stale connection-time snapshot.
  - Blocked by: `mcp-bridge-tools-snapshot-at-connect` — The tool catalog and server instructions are populated once during `connect()` and never refreshed — there is no mechanism to pick up tools added after the initial handshake.
- **governance-and-dialectics-have-verified-references:** Complete governance across topology, source, and traceability with dually-grounded dialectical operations achieves verified reference integrity at all node ID boundaries — governance completeness and dialectical assurance are not only independently established but reference-verified.
  - Blocked by: `issue-126-reference-validation-audit` — Issue #126: Audit all node ID reference boundaries for validation — three specific boundaries do not establish coverage of every boundary
- **governance-topology-is-reference-verified:** Rich governance's topology-complete transitions and metadata-governed bidirectional modifications have fully verified reference integrity at all system boundaries — no node ID reference bypasses validation at any boundary — when the reference validation audit confirms coverage at the three identified boundary gaps (issue #126).
  - Blocked by: `issue-126-reference-validation-audit` — Issue #126: Audit all node ID reference boundaries for validation — three specific boundaries do not establish coverage of every boundary
- **review-achieves-verified-fault-tolerance:** The review pipeline's scoped mutation-safe operation combined with uniform fail-safe output achieves verified fault tolerance across all failure modes — batch failures, missing antecedent references, and malformed LLM responses.
  - Blocked by: `issue-122-review-fault-tolerance-audit` — Issue #122: Audit review module for unhandled failure modes — three specific handlers do not establish coverage of all failure modes
- **revision-has-code-enforced-derivation-constraints:** The deterministic traceable revision system with complete dialectical semantics achieves fully code-enforced derivation quality — every constraint including minimum antecedent requirements is validated programmatically, not relying solely on LLM prompt instructions for structural invariants.
  - Blocked by: `derive-min-antecedents-is-prompt-only` — The minimum-2-antecedents rule for derived beliefs is enforced only by the LLM prompt instructions, not validated in code by `validate_proposals`.
- **rich-governance-has-verified-evolution-tolerance:** The system's rich governance —

deterministic, exception-safe, and source-grounded — combined with gap-free lifecycle source coverage achieves verified evolution tolerance at all system boundaries when the evolution tolerance audit (issue #121) confirms that all boundaries have documented forward-compatibility, completing the governance guarantee across the format/schema evolution dimension.

- Blocked by: **issue-121-evolution-tolerance-audit** — Issue #121: Audit evolution tolerance at all system boundaries — not all boundaries have documented forward-compatibility mechanisms
- **rich-governance-spans-all-backends:** The system’s exception-safe rich lifecycle governance — deterministic, lifecycle-complete, and spanning revision through source integrity — operates identically across all storage backends, achieving full behavioral parity between SQLite and PostgreSQL for metadata-enabled lifecycle management.
  - Blocked by: **pgapi-partial-api-coverage** — PgApi implements core operations (add/retract/assert/search/nogood/explain) but defers simulation, dialectics, namespace support, import/export, and maintenance operations to future work
- **verified-revision-completeness-at-all-reference-boundaries:** The deterministic lifecycle-complete architecture achieves verified uniform revision completeness — every belief case handled uniformly within predictable monitored state trajectories AND every node ID reference crossing a system boundary validated against the actual network — eliminating the possibility of revision operations acting on phantom references.
  - Blocked by: **issue-126-reference-validation-audit** — Issue #126: Audit all node ID reference boundaries for validation — three specific boundaries do not establish coverage of every boundary

## New Negative IN Beliefs

- **apply-dedup-plan-collects-errors-not-raises:** `apply_dedup_plan` collects errors into `result["errors"]` rather than raising, allowing partial application — one missing node does not block processing of the remaining dedup plan
- **edge-case-uniformity-reinforces-complete-negative-semantics:** Uniform handling of all semantic edge cases — vacuous premises, asymmetric absence, empty antecedents — reinforces the completeness and recoverability of negative semantics: every edge case within the negation lifecycle (missing outlist nodes treated as OUT, challenge of already-justified nodes, recovery from outlist defeat) is handled by the same minimal evaluation rules that ensure reversibility.
- **lifecycle-governance-achieves-gap-free-source-coverage:** Metadata-enabled lifecycle governance with deterministic source integrity and exception-safe recoverability achieves truly gap-free source coverage — every source file is verified, every lifecycle decision is grounded in verifiable source state, and no silent source gap undermines governance decisions about staleness or retraction.
- **source-integrity-and-governance-form-closed-loop:** Source integrity enables lifecycle governance by unifying determinism, exception safety, and lifecycle management into a single pipeline, while lifecycle governance achieves gap-free source coverage — forming a self-reinforcing closed loop where source integrity grounds the governance that in turn ensures no source verification gap exists.

## Critical Watch List

### Gated (blocked)

- **storage-is-fully-production-grade-across-backends:** Both storage backends achieve fully production-grade operation — concurrent access optimization (WAL mode, derived FTS5 indexes), equivalent safety guarantees (atomic isolated mutations through backend-appropriate mechanisms), and multi-tenant isolation — when both backends provide complete API coverage.
  - Blocked by: **pgapi-partial-api-coverage** — PgApi implements core operations (add/retract/assert/search/nogood/explain) but defers simulation, dialectics, namespace support, import/export, and maintenance operations to future work
- **verified-correctness-extends-to-all-backends:** Verified production correctness — spanning all belief origins with deterministic state trajectories and full maintenance loop observability for indefinite independent verification — extends identically across both SQLite and PostgreSQL storage backends with no backend-specific safety gaps.
  - Blocked by: **pgapi-partial-api-coverage** — PgApi implements core operations (add/retract/assert/search/nogood/explain) but defers simulation, dialectics, namespace support, import/export, and maintenance operations to future work

### Active Issues

- **llm-prompt-always-via-stdin:** Prompts are passed to LLM subprocesses via `subprocess.run(input=...)` never as command-line arguments — avoiding shell injection and argument length limits.

### Statistics

- **Total beliefs:** 1101
- **IN:** 711
- **OUT:** 390
- **Derived:** 619
- **Gated OUT (all):** 32
- **Negative IN (all):** 48
- **New beliefs this run:** 145
- **New gated OUT:** 9
- **New negative IN:** 4